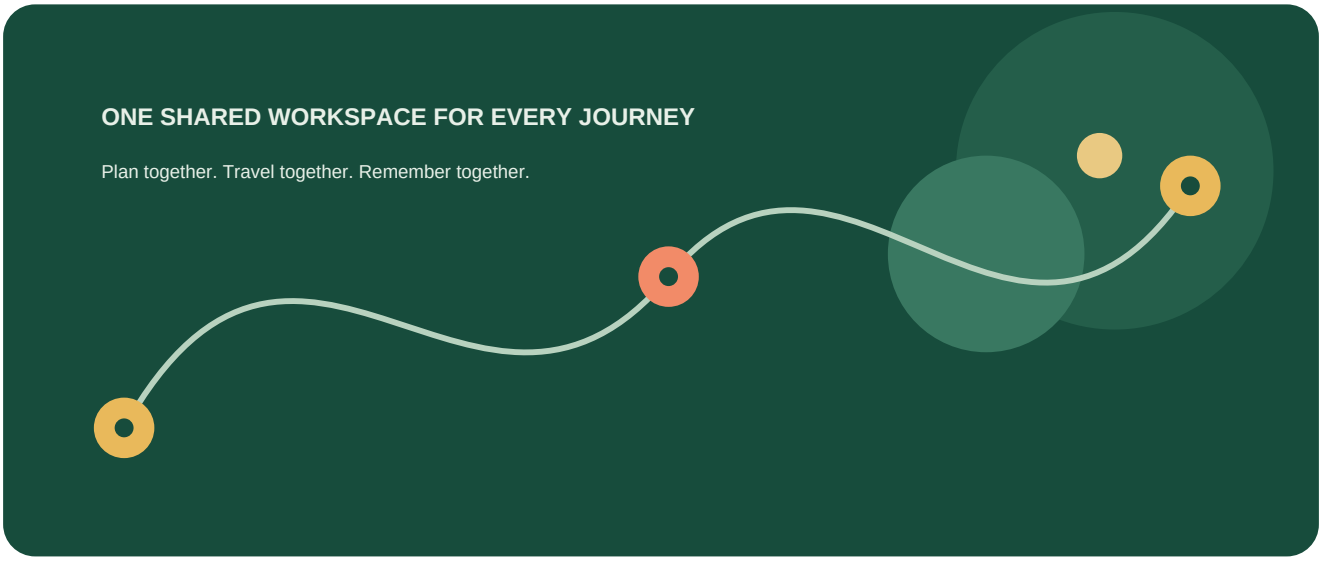
Emergent Build Guide

The end-to-end workflow and copy-ready master prompt for building a modern, interactive TripOS frontend.



YOUR TARGET
A premium responsive web app with a complete demo journey, realistic mock content and a frontend architecture ready for later Claude Code backend integration.

Prepared from the TripOS PRD v0.1 provided for this project. Treat the PRD as the product source of truth; this guide translates it into an Emergent-focused frontend build plan.

Contents

1. Quick start: how to use this PDF
2. Product brief and MVP scope
3. Recommended visual direction
4. App information architecture
5. Build workflow for Emergent
6. Frontend architecture and backend handoff
7. Copy-ready master prompt for Emergent
8. Follow-up prompts for quality and refinement
9. Acceptance checklist
10. Handoff to Claude Code

1. Quick start: how to use this PDF

Attach this PDF to Emergent and give it the instruction in the callout below. Ask Emergent to read the full guide before generating code. Then proceed through the staged workflow in Section 5 instead of requesting an unreviewed all-at-once build.

FIRST MESSAGE TO EMERGENT

Read the attached TripOS Emergent Build Guide completely and use it as the source of truth for this task. Follow its build workflow and the exact master prompt in Section 7. Build a frontend-first, responsive and interactive TripOS web application. Keep backend, database, production authentication, storage and microservices out of scope. Use realistic local mock data behind replaceable service interfaces so I can integrate the backend later using Claude Code.

The goal

- Generate a real frontend application, not just a static landing page or clickable mockup.
- Deliver the core MVP journey: demo entry -> dashboard -> create trip -> trip workspace -> manage itinerary, expenses, tasks and packing.
- Use the same visual system across desktop, tablet and mobile.
- Keep code organized, typed and ready for future REST API integration.

How to work with Emergent

- Give it this PDF as the brief.
- Start by asking for the design system and app shell plus the dashboard/trips foundation.
- Review the result and ask for corrections before expanding to more modules.
- Continue with the stage-by-stage prompts in Section 8.
- Export/sync the source code and run the handoff checklist before using Claude Code.

KEEP THIS BOUNDARY CLEAR

Emergent is being asked to build the frontend experience. Claude Code will be used afterward for backend, database and production integrations. Do not let Emergent invent a production backend or add database dependencies that make the frontend harder to export.

2. Product brief and MVP scope

TripOS is an all-in-one collaborative travel workspace designed to simplify every stage of a group trip. It aims to replace fragmented coordination across chat apps, spreadsheets, expense tools, cloud drives, maps and notes. It is not intended to become another travel booking platform; its focus is coordination.

User problems

- Endless group messages and date conflicts.
- Unclear budgets, expense splits and settlements.
- Lost booking confirmations and scattered documents.
- Unclear task ownership and forgotten packing items.
- No shared itinerary, group workspace or organized memories after travel.

Product principles

- Simple, collaborative, fast, reliable and privacy-conscious.
- AI can enhance workflows but essential features must work without AI.
- Mobile-first responsive web app; native Android/iOS are not part of the MVP.
- Build incrementally. Make each delivered phase usable.

MVP capabilities from the PRD

Area	MVP behavior to represent in the frontend
Accounts / users	Demo sign-in or demo entry, profile and user menu; real auth deferred
Trips	Create trip, invite members, roles represented in UI, dashboard, archive
Decisions	Destination/date/activity voting UI with local interactions
Itinerary	Day-wise planner, timeline, activities, locations and notes
Responsibilities	Tasks, assignments, priority, due dates and completion
Budget / expenses	Expense entries, equal/custom split UI, balances and settlement summary
Packing	Personal/shared checklists, categories and progress
Documents	Shared ticket/confirmation/insurance file area; simulated file interactions
Chat	Trip chat UI, message entry, polls, pinned items and attachments simulated
Maps	Saved places and meeting points; map can be a clearly labelled mock if no map provider
Archive / memories	Completed trip view, gallery and trip recap/replay concept

The PRD also mentions future enhancements such as AI itinerary assistance, receipt OCR, voice assistance, community itineraries, public templates, business/family modes, recommendations and integrations. Treat these as future or exploratory, not MVP requirements.

3. Recommended visual direction

Recommended design: a premium, friendly travel SaaS interface combining the clarity of Linear, the organization of Notion and the visual warmth of destination-led travel products. This is a design recommendation for the build, not an additional product requirement in the PRD.

Design tokens

Token	Hex	Purpose
Forest green	#174C3C	Primary actions, selected navigation, key accents
Soft sage	#E7EFE9	Soft panels and selected states
Warm coral	#F28B68	Secondary accent and moments of emphasis
Warm off-white	#F8F8F5	Main application background
White	#FFFFFF	Cards and primary surfaces
Charcoal	#202824	Primary text
Muted gray	#7A827D	Secondary text
Neutral border	#DDE4DE	Subtle dividers and outlines

Typography, surfaces and motion

- Typography: Inter or Geist; use a deliberate hierarchy and readable line lengths.
- Cards: mostly white surfaces, subtle borders, restrained shadow and 12-18px radius.
- Icons: consistent Lucide outline icon family.
- Images: high-quality destination imagery on trip cards, trip cover areas and memories. Do not use random stock images as decoration.
- Motion: purposeful subtle hover, modal, tab and progress transitions. Respect reduced-motion settings.
- Accessibility: strong contrast, keyboard access, labels, visible focus states and touch-sized controls.

AVOID

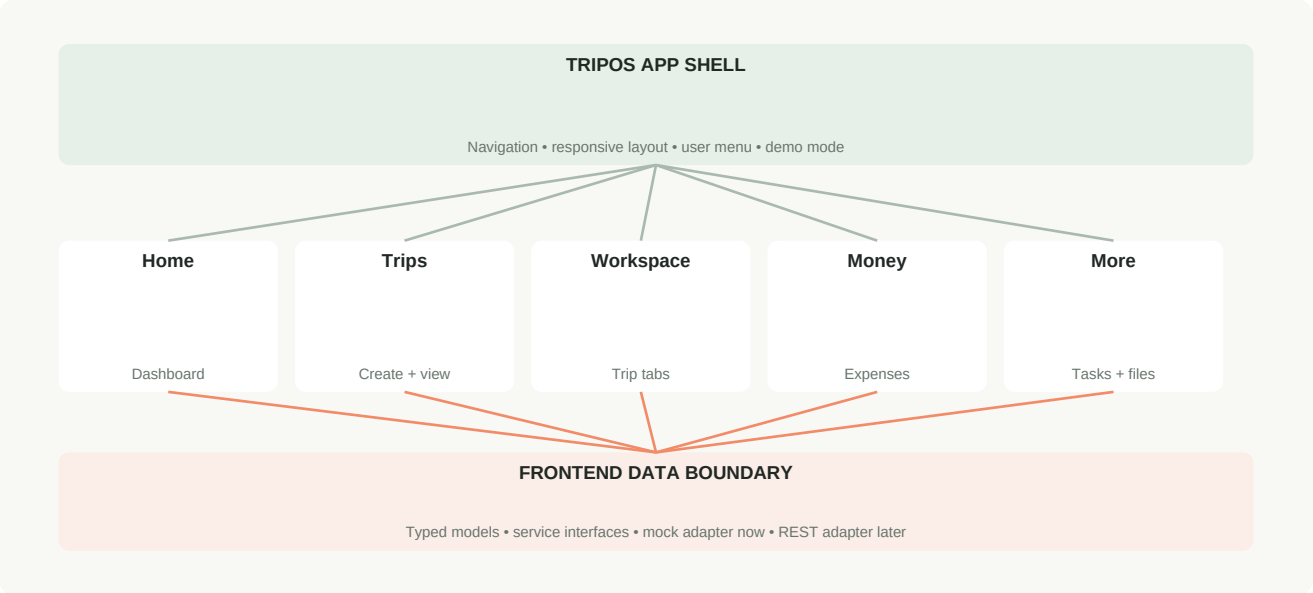
Avoid generic admin templates, cluttered dashboards, neon gradients, excessive glassmorphism, oversized cards everywhere, inconsistent icon styles and animation that slows down routine tasks.

Visual cues for Emergent

- Dashboard: calm, information-rich overview with one prominent next-trip card, quick actions and compact activity panels.
- Trips: editorial cards with attractive destination photos, dates, member avatars and status.
- Trip workspace: destination cover/header, contextual tabs and a clear day-by-day itinerary timeline.
- Expenses: clean Splitwise-inspired breakdown with balances and an easy-to-scan transaction list.
- Mobile: intentionally redesigned compact navigation, stacked cards, bottom navigation and responsive forms.

4. App information architecture

Use one coherent shell for the first frontend delivery. Organize features into clear modules even if the environment cannot create separately deployed micro-frontends at this stage.



Navigation item	What belongs here
Overview	Personalized greeting, upcoming trips, activity, quick actions
My Trips	Trip grid/list, search, filters, create and archive
Explore	Optional visual discovery demo; avoid booking functionality
Expenses	Personal/group expense overview and transactions
Tasks	Tasks across trips and pending responsibilities
Memories	Completed trip photos, albums and recap
Shared Documents	Documents across trips, categorized files
Settings	Profile, preferences, currency, notifications and privacy

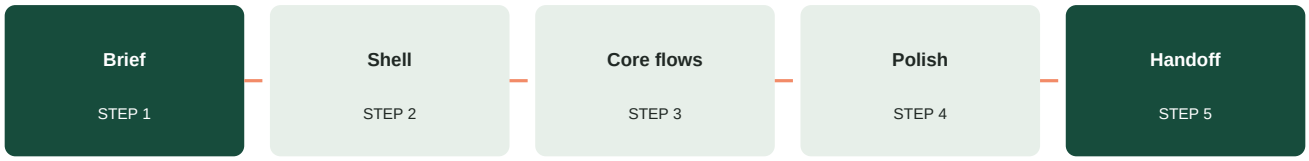
Trip workspace tabs

- Overview - trip summary, members, countdown, recent changes and quick actions.
- Itinerary - day selector, timeline and activities.
- Expenses - transactions, balances and settlement.
- Tasks - assignments and progress.
- Packing - personal and shared checklists.
- Documents - shared tickets and confirmations.
- Map - saved places and meeting points.
- Chat - trip conversation and polls.
- Memories - gallery, albums and recap.

Keep the primary desktop navigation persistent. On mobile, use compact top navigation plus bottom navigation for the most-used destinations and a drawer/sheet for secondary pages. Do not shrink the desktop sidebar into an unusable mobile

column.

5. Build workflow for Emergent



Stage	Build	Review before continuing
0. Inspect	Read this PDF; confirm assumptions; identify supported stack and limitations.	No surprise backend, data store or auth choices.
1. Foundation	Design tokens, typography, buttons, forms, cards, dialogs, toasts, navigation shell and responsive behavior.	Consistent, polished shell at desktop and mobile widths.
2. Core trip loop	Demo entry, dashboard, My Trips, trip creation wizard, trip workspace overview.	Can create a trip locally and land in its workspace.
3. Planning	Itinerary timeline, tasks, packing and documents.	Forms and actions work; local state updates the UI.
4. Group finances	Expense create/edit, equal/custom split, member balances and settlement summary.	Calculations validated with sample scenarios.
5. Collaboration	Chat, polls, saved places/map view, memories/gallery and archive.	Mock behaviors are clearly simulated, not misrepresented as live services.
6. Polish	Responsive QA, accessibility, loading/empty/error states, interaction pass and bug fixes.	No dead primary buttons, broken routes or layout overflow.
7. Export	Document components, models, mock data, service boundaries, setup and run instructions.	Repo runs locally; handoff checklist complete.

Recommended first milestone

DO THESE FIRST

Build the application shell, dashboard, My Trips, create-trip flow and the trip workspace overview. These screens establish the product identity and the primary user journey. Do not spend the first iteration polishing lower-priority pages while the core trip loop is incomplete.

Demo experience

Use a clearly labelled demo entry option so the reviewer can enter the product without configuring real authentication. Populate the app immediately with realistic sample data: at least 3 trips across upcoming/ongoing/completed states, 4-6 members, 10 itinerary activities, 10 expenses, 10 tasks, 15 packing items, 5 documents and 10 chat messages. Default the sample currency to INR, while allowing other currencies in the UI.

6. Frontend architecture and backend handoff

The PRD specifies a future TypeScript architecture with Next.js/React, Tailwind, shadcn/ui, NestJS services, PostgreSQL/Prisma, Clerk or Auth.js, Socket.IO, Redis, BullMQ, Cloudflare R2, Mapbox/Google Maps, PostHog and Sentry. It also describes an Nx monorepo and separate app/service folders. These are long-term architecture intentions. The Emergent task should not attempt to scaffold all of that infrastructure unless the environment makes it necessary and explicitly supports it.

Recommended frontend boundary

Layer	Responsibility now	Later replacement / connection
UI / pages	Screens, shared components, responsive shell	Keep reusable; connect to API DTOs
Feature state	Local state and client-side validation	Server state / cache / mutation hooks
Domain types	User, Trip, Member, Activity, Expense, Task, Document	Align with Claude Code backend DTOs
Service interfaces	TripService, ExpenseService, TaskService, etc.	NestJS REST endpoints
Mock adapters	Realistic in-memory/localStorage demo data	Replace with real API adapters
Integration config	Environment variable placeholders and API base URL config	Real auth, API gateway, storage, maps

Suggested source structure

```
src/
  app/                    # routes and app composition
  components/
    ui/                   # buttons, inputs, dialogs, cards
    layout/               # shell, sidebar, topbar, mobile nav
    shared/               # avatars, badges, empty states
  features/
    dashboard/
    trips/
    itinerary/
    expenses/
    tasks/
    packing/
    documents/
    maps/
    chat/
    memories/
    profile/
  services/
    api/                  # API client and interfaces
    mock/                 # mock implementations and seed data
  hooks/
  types/
  lib/
  styles/
```

Important boundaries

- Do not embed mock data directly inside visual components. Centralize it in mock repositories/adapters.
- Use typed models and feature-specific service interfaces. Keep UI independent from how data is loaded.
- Do not add production database migrations, Prisma schema, NestJS microservices or Docker infrastructure to this frontend task.
- Do not claim that simulated chat, uploads, maps, invitations or notifications are live. Label or document the simulation where relevant.
- Do not commit API keys or secrets. Use environment placeholders only.
- If Emergent cannot use the exact requested framework, state the actual stack selected and keep the app exportable and maintainable.

7. Copy-ready master prompt for Emergent

HOW TO USE

Copy everything between START MASTER PROMPT and END MASTER PROMPT into Emergent. Attach this PDF as well. The prompt is intentionally explicit so the builder understands both what to create and what not to create.

START MASTER PROMPT

PROJECT: TRIPOS - COLLABORATIVE GROUP TRAVEL WORKSPACE

READ THE ATTACHED TRIPOS EMERGENT BUILD GUIDE BEFORE WRITING CODE. Treat it as the product and design brief for this task. Follow its workflow, frontend boundaries, visual direction and acceptance criteria.

1. YOUR ROLE AND OUTCOME

Act as a senior SaaS product designer, UX architect and frontend engineer. Build a polished, production-quality FRONTEND for TripOS: an all-in-one collaborative workspace for group trips. The outcome must be a working, responsive web application with realistic demo data and meaningful interactions - not just a landing page, static prototype or generic admin dashboard.

Product positioning: "Everything your group needs for a trip, in one place." TripOS helps a group plan together, travel together, stay organized together and preserve memories. It is NOT a travel booking platform. Focus on coordination, not reservations.

2. SCOPE BOUNDARY - CRITICAL

This task is frontend-first. I will integrate backend services and database later using Claude Code.

DO NOT build or provision a production backend, database, Prisma schema, NestJS microservices, API gateway, Redis, queues, production auth, payment processing, cloud storage or infrastructure-as-code.

DO NOT introduce a managed backend/database provider as a hidden dependency.

Use realistic mock data behind typed service interfaces and a replaceable mock data adapter. Use local client state and localStorage persistence where appropriate for the demo. The app must remain usable without external credentials.

Do not make simulated features appear live. Clearly treat chat, invitations, uploads, location and notifications as demo/local behavior.

3. PRODUCT PRINCIPLES

- Simplify group trip planning and coordination.
- Centralize trip information, decisions, money, responsibilities, documents, communication and memories.
- Keep essential workflows usable without AI.
- Favor simplicity, collaboration, speed, reliability, privacy and delightful UX.
- Responsive web app, mobile-first. Native mobile apps are out of scope.
- Build incrementally and preserve working functionality when refining.

4. TECHNICAL PREFERENCES

Prefer React + TypeScript. Use Next.js App Router if the environment supports it cleanly. Tailwind CSS and shadcn/ui are preferred for styling/components; Lucide icons; Framer Motion for restrained motion; Recharts for useful charts; React Hook Form and Zod for forms if available.

Do not force unsupported libraries or an Nx/MFE monorepo if the Emergent environment cannot support them. If a different stack is required, explain the choice and keep the resulting source organized, typed where possible, and easy to export. Do not sacrifice a working build to imitate an unavailable architecture.

5. VISUAL DESIGN SYSTEM

Create a premium, friendly travel SaaS identity inspired by Linear's clarity, Notion's organization and the visual warmth of travel experiences. Do not copy any existing product screen.

Use these starting tokens:

- Primary forest green: #174C3C
- Soft sage: #E7EFE9
- Warm coral accent: #F28B68
- Warm off-white background: #F8F8F5
- White surfaces: #FFFFFF
- Primary text: #202824
- Secondary text: #7A827D
- Borders: #DDE4DE

Typography: Inter or Geist.

Components: subtle borders, restrained shadows, consistent spacing, mostly 12-18px corner radii, clean typography, coherent Lucide iconography, quality destination photography where it adds value.

Avoid excessive gradients, excessive glassmorphism, neon colors, huge cards everywhere, clutter, decorative-only widgets and generic admin template styling. Maintain strong contrast and accessible focus states.

6. APP SHELL AND NAVIGATION

Build one cohesive application shell for this first delivery. Desktop: persistent left sidebar with TripOS logo, Overview, My Trips, Explore (non-booking discovery/demo), Expenses, Tasks, Memories, Shared Documents and Settings. Include a prominent Create Trip action. At the bottom show the current demo user, avatar and profile/help menu. Main area has contextual page title, relevant actions, notifications access and breadcrumbs where useful.

Mobile: do not simply shrink the desktop UI. Use a compact top header, bottom navigation for high-use areas and a drawer/sheet for secondary navigation. Make controls touch-friendly and avoid horizontal overflow.

7. REQUIRED SCREENS AND BEHAVIOR

A. Demo entry / welcome:

- TripOS branding and concise product positioning.
- Simulated sign-in/sign-up screens if needed.
- A clear "Explore demo" or equivalent entry that opens a populated demo workspace without real authentication.

B. Overview dashboard:

- Personalized greeting and next trip emphasis.
- Upcoming/active trip cards with destination images, trip name, destination, dates, member avatars/count, status and countdown.

- Recent activity feed, pending tasks, recent expenses and quick actions.
- Quick actions: Create trip, Join trip (simulated), Add expense, Create task and Open itinerary.
- Useful information density: clean and scannable, not crowded.

C. My Trips:
- Grid/list view toggle, search, filters for upcoming/ongoing/completed, create, open and archive actions.
- Cards include destination imagery, trip name, destination, dates, avatars, activity count, budget summary and status.

D. Create Trip wizard:
- Step 1: name, destination, cover image, description.
- Step 2: start and end dates with validation.
- Step 3: estimated group budget and currency.
- Step 4: invite members by email (simulated/local).
- Step 5: review and create.
- Show progress; support back/next; validate required fields.
- On completion, persist the new trip in local demo state and route to that trip's workspace.

E. Trip workspace:
- Header with destination, dates, cover image, member avatars, countdown, invite action (simulated) and trip settings.
- Contextual tabs: Overview, Itinerary, Expenses, Tasks, Packing, Documents, Map, Chat and Memories.
- Each tab has a distinct useful layout, not just a heading and empty placeholder.

F. Itinerary builder:
- Day selector and day-wise timeline.
- Add/edit/delete activity; title, time, duration, type, location, description, estimated cost, notes and assigned members.
- Distinguish travel, activities, meals, accommodation and free time with restrained labels/icons.
- Drag-and-drop reorder if supported; otherwise provide explicit move up/down controls.
- Include useful empty states.

G. Expense management:
- Summary of total trip spend, user's contribution, outstanding balance, spending by category and member balances.
- Recent transaction list and a useful chart.
- Add/edit expense: title, amount, currency, paid by, participants, equal split/custom split, category and date.
- Categories: accommodation, food, transport, activities, shopping and miscellaneous.
- Implement and test local equal/custom split calculations. Show a clear settlement summary of who owes whom. Do not claim payments have been made.

H. Tasks and responsibilities:
- Create/edit/complete task, assign member, due date, priority and status.
- List or Kanban view; filter by member and status.

I. Packing:
- Personal and shared packing lists, categories, custom items, assignment, completion toggle and progress.
- Example categories: clothing, electronics, documents, toiletries, medicines and travel essentials.

J. Shared documents:
- Categories for tickets, hotel confirmations, insurance, emergency contacts and other documents.
- Show filename, type, upload date and uploader; provide preview/download/delete UI where practical.
- Simulate uploads locally; do not connect real storage or imply secure cloud storage exists.

K. Map / saved places:
- Show destination context, saved places, hotel, restaurant, activity and meeting point markers/details.
- If a working map provider is unavailable, use an attractive, clearly labelled map placeholder with location cards; do not pretend it provides live navigation or real-time location.

L. Group chat:
- Trip-specific message list, avatars, timestamps, message input, simulated image/file attachment UI, poll cards, pinned messages and location cards.
- Simulate locally and make the demo interaction work.

M. Memories:
- Shared photo grid, albums, timeline, photo preview modal and completed trip recap/statistics.
- Use appropriate placeholder imagery. Do not invent actual user-uploaded memories.

N. Profile and settings:
- Demo profile, avatar, personal details, currency preference, notification preferences, privacy and appearance settings.

8. MOCK DATA REQUIREMENTS

The demo must look populated immediately. Seed at least:
- 3 trips covering upcoming, ongoing and completed states.
- 4-6 trip members.
- 10 itinerary activities.
- 10 expense transactions.
- 10 tasks.

- 15 packing items.
- 5 shared document examples.
- 10 chat messages.

Use realistic Indian and international destinations. Default demo currency to INR but allow other currencies in the UI. Make sample content internally consistent (dates, totals, participants and expense splits).

9. INTERACTION REQUIREMENTS

All major visible buttons must do something. Implement working navigation, tabs, trip creation, search/filter/sort, dialogs, dropdowns, form validation, toast feedback, checklist toggles, task completion, member selection, local expense math, day selection and trip archive. Use shared application state so a created trip or expense appears in relevant screens. Persist demo state in localStorage when practical, with a reset-demo option. Avoid dead buttons, fake links and controls that only look interactive.

10. FRONTEND CODE ORGANIZATION

Keep UI, state and data access separated. Prefer a feature-oriented structure such as:

src/app/
src/components/ui/
src/components/layout/
src/components/shared/
src/features/dashboard/
src/features/trips/
src/features/itinerary/
src/features/expenses/
src/features/tasks/
src/features/packing/
src/features/documents/
src/features/maps/
src/features/chat/
src/features/memories/
src/features/profile/
src/services/api/
src/services/mock/
src/hooks/
src/types/
src/lib/
src/styles/

Use typed domain models and service interfaces such as TripService, ExpenseService, ItineraryService, TaskService, PackingService and DocumentService. UI components must not import seed data directly. The mock adapter should be replaceable by a future REST adapter without rewriting the screens. Document where real API calls will be connected.

11. RESPONSIVE, ACCESSIBILITY AND QUALITY

Verify at approximately 1440px, 1280px, 768px, 390px and 375px. No horizontal overflow. Use semantic structure, labels, keyboard navigation, visible focus states, meaningful contrast and touch-sized controls. Respect prefers-reduced-motion. Include loading/skeleton, empty, error and success states as appropriate. Test core flows after each milestone.

12. REQUIRED BUILD ORDER

1) Inspect requirements and confirm supported stack; state any limitations before implementation.
2) Create design tokens and reusable UI primitives.
3) Build responsive shell and navigation.
4) Build demo entry, dashboard and My Trips.
5) Build Create Trip flow and Trip Workspace overview.
6) Build itinerary, tasks, packing and documents.
7) Build expenses and validate calculations.
8) Build chat, map/saved places, memories and archive.
9) Run responsive/accessibility/interaction QA; fix issues without removing functionality.
10) Provide source structure, setup/run instructions, mock data notes, API integration points and known limitations.

13. ACCEPTANCE CRITERIA

The build is accepted only when:

- It looks like a premium cohesive SaaS product, not a generic admin dashboard.
- A user can enter demo mode and reach a populated dashboard.
- A user can create a trip and open its workspace.
- Core trip tabs have useful UI and working local interactions.
- Expense equal/custom split calculations work for tested sample cases.
- Search, filters, navigation, tabs, forms and main buttons work.
- Desktop, tablet and mobile layouts are usable with no horizontal overflow.
- Mock data is centralized behind service abstractions.
- No production backend/database/auth/storage dependency has been introduced.
- The project can be exported and run using documented steps.

14. HOW TO COLLABORATE DURING THE BUILD

```
Do not try to generate every module in one opaque pass. Work in the staged order above. After each stage, summarize what was implemented, what is functional, what remains and any assumptions. Ask for clarification only when a decision materially blocks implementation; otherwise make a sensible reversible frontend assumption and document it. Preserve existing working UI and interactions when making refinements.

FINAL INSTRUCTION: Prioritize exceptional UI/UX quality and a coherent end-to-end demo journey over feature count. Make TripOS feel like a product people would genuinely enjoy using to organize their next group trip.
```

END MASTER PROMPT

8. Follow-up prompts for quality and refinement

A. Review the dashboard

Copy this after Emergent has built the dashboard:

PROMPT

Review the current TripOS dashboard as a senior SaaS product designer. Preserve the established design tokens and working functionality. Improve information hierarchy, spacing, trip card imagery, content density, alignment, typography, responsive behavior, and empty/loading states. Make the page feel premium and useful rather than generic. Do not add a new visual direction or remove working features.

B. Audit interactions

PROMPT

Perform an interaction audit of the entire current TripOS frontend. Click/test every visible primary action, navigation item, tab, form, dropdown, checkbox and modal. List dead controls, broken routes, missing validation and inconsistent state. Fix the issues. Verify that creating a trip, adding an expense, completing a task and checking packing items updates the relevant UI and persists in the demo session. Do not introduce backend services.

C. Mobile refinement

PROMPT

Audit TripOS at 390px and 375px widths. Redesign any desktop-derived layouts that feel cramped. Check bottom navigation, safe spacing, cards, tables, forms, dialogs, tab overflow, image crops and touch targets. Remove horizontal overflow. Preserve all functionality and the existing visual identity.

D. Visual consistency pass

PROMPT

Review all TripOS pages as one product. Standardize spacing, typography, colors, borders, radii, icon sizes, button hierarchy, form states, badges, modals, toasts and empty states. Remove inconsistent one-off styles. Do not change the product scope or break working interactions.

E. Prepare code for Claude Code

PROMPT

Prepare the frontend for backend integration by Claude Code. Document the domain types, service interfaces, mock adapters, local persistence, state boundaries, route structure, environment variables and each likely REST API integration point. Ensure pages do not directly import seed data. Do not implement a backend or database. Provide a concise integration map and setup/run instructions.

9. Acceptance checklist

Use this checklist before calling the Emergent frontend milestone complete.

Check	Done when
Demo entry	Can enter demo without real authentication and see populated data.
Dashboard	Upcoming trips, activity, tasks, expenses and quick actions are coherent.
Trip creation	Wizard validates input, creates a local trip and routes to workspace.
Trip workspace	Tabs navigate; each MVP tab has meaningful content.
Itinerary	Can add/edit/delete/reorder or move activities; day switching works.
Expenses	Equal/custom calculations are correct for test cases; balances reconcile.
Tasks and packing	Can assign/complete tasks and toggle packing items; progress updates.
Documents/chat	Simulated interactions are clear; no false claim of live sync/storage.
Responsive	No overflow at 375, 390, 768, 1280 and 1440 widths.
Accessibility	Labels, keyboard focus, contrast and reduced-motion behavior considered.
Code boundary	Mock data centralized; typed service interfaces; no production backend/database dependencies.
Export	Source can be exported and run from documented steps.

10. Handoff to Claude Code

Once the frontend has passed the checklist, export or sync the source to GitHub and keep a stable baseline commit. Then ask Claude Code to inspect the entire codebase before implementing backend integrations.

Suggested handoff sequence

- Export/sync source code and create a clean development branch.
- Run the app locally and verify the Emergent version matches the exported source.
- Ask Claude Code to map routes, domain types, local state, mock adapters and service interfaces before changing code.
- Agree on API contracts and authentication strategy against the TripOS PRD.
- Replace one mock service at a time with real REST endpoints; retain demo mode where useful.
- Add backend/database architecture incrementally rather than coupling all services in one integration step.

Prompt for Claude Code handoff

```
Inspect this TripOS frontend before making changes. Do not immediately refactor it. First map the routes, feature modules, shared UI, domain types, mock data, state management, service interfaces, localStorage usage and current build/run commands. Identify the exact points where real API calls can replace mock adapters. Compare the frontend assumptions with the TripOS PRD. Report mismatches, missing contracts, risks and a staged integration plan. Preserve the visual design and existing working demo journey. Do not create or modify database schemas until we agree on the API contracts and data model.
```

FINAL PRODUCT PRINCIPLE

TripOS should feel like the shared home base for a journey: organized enough to reduce group chaos, visual enough to inspire the trip, and simple enough that every member can use it.